ARKITEKTURDOKUMENT & BYGGPLAN (ROADMAP)

Anonym CV-Matchning: Från Pilot till SaaS

Komplett teknisk specifikation för ombyggnad av `pilot(2).py` till en skalbar, säker och GDPR-anpassad webbapplikation

---

1. Bakgrund & Varför vi bygger om

Den nuvarande prototypen (`pilot(2).py`) är byggd som en monolitisk Streamlit-fil. Den har bevisat affärsvärdet i anonym, kompetensbaserad CV-matchning, men har tekniska begränsningar som måste byggas bort innan tjänsten kopplas till en publik hemsida med betalande kunder:

- **Minneshantering & Dataförlust:** Alla uppladdade PDF-filer och analysresultat lagras direkt i serverns RAM-minne (`st.session_state`). Om användaren råkar uppdatera webbläsaren (F5) raderas hela analysen, och vid många samtidiga användare riskerar servern att krascha av minnesbrist.
- **Kontohantering & Kostnadskontroll:** Alla användare delar på ett enda lösenord (`APP_PASSWORD`). Det saknas individuella kundkonton, spårning av förbrukning och spärrar (limits) som förhindrar att en enskild kund förbrukar obegränsat med AI-krediter.
- **Bräcklig PDF-läsning & JSON-tolkning:** Prototypen använder `PyPDF2` som ofta tappar textstruktur i moderna flerkolumns-CV:n, samt manuell sträng-splittning (`split("```json")`) som orsakar systemfel om AI-modellen lägger till text utanför JSON-blocket.
- **Onödigt hög AI-kostnad i tvättsteget:** Idag görs två tunga anrop till Claude Sonnet per CV (både för anonymisering och för bedömning).

2. Målbild, Hemsideskoppling & Teknikstack (EU)

Applikationen byggs om enligt **Alternativ A (Fullskalig SaaS-stack)** med en tydlig separation mellan användargränssnitt (Frontend) och AI-/affärslogik (Backend). Samtliga servrar och databaser placeras inom EU (Stockholm/Frankfurt) för full GDPR-efterlevnad.

Koppling till hemsidan

Genom att bygga frontend i **Next.js** får vi maximal flexibilitet för hemsidan:

- **Rekommenderat upplägg (Allt-i-ett):** Både den publika marknadshemsidan (`dindomän.se`), inloggningen (`dindomän.se/login`) och själva applikationen (`dindomän.se/dashboard`) bor i samma Next.js-projekt. De delar design och ger omedelbar navigering.
- **Alternativt upplägg (Subdomän):** Om marknadshemsidan byggs i ett externt verktyg (t.ex. Webflow eller Framer) pekas knappen "Logga in" direkt till `app.dindomän.se` . Kodstrukturen stödjer båda alternativen utan ombyggnad.

Vald teknikstack & Hosting

Systemdel	Teknikval	Hosting (EU)	Syfte & Fördel
Frontend (Hemsida & App)	Next.js (React), TypeScript, Tailwind CSS	Vercel eller Render (Frankfurt)	Snabb SaaS-känsla, skräddarsydd design för Leaderboard, kandidatvy och kvotöversikt.
Backend (API & AI-motor)	FastAPI (Python 3.11+), Pydantic, Asyncio	Render (Frankfurt)	Hanterar PDF-till-Markdown, parallella AI-anrop, kvotkontroll och Excel-export.
Databas, Inloggning & Filer	Supabase (PostgreSQL, Auth, Storage)	Supabase EU (Stockholm / Frankfurt)	Färdig kontohantering, Row Level Security (RLS) och krypterad fillagring med bäst-före-datum.
AI-leverantör	Anthropic API (Haiku 4.5 + Sonnet 4.5)	Commercial API (DPA / Zero Data Retention)	Ingen träning på kunddata, stöd för Prompt Caching och låsta JSON-scheman.

3. Optimering av PDF- & AI-Pipeline

För att maximera hastighet, sänka token-kostnaden och garantera anonymitet ersätts den gamla textextraktionen med en lokal Markdown-konvertering följt av en tvåstegspipeline i AI-motorn.

Varför lokal PDF-till-Markdown vinner över direkt PDF-uppladdning

1. **60–80 % lägre token-kostnad:** Om en PDF skickas direkt till Claudes PDF-API betalar man både för texten och för att Claude läser varje sida som en bild (1 500–3 000 extra tokens per sida). Lokal konvertering till Markdown (`.md`) kostar 0 kr och skickar enbart ren, strukturerad text.
2. **Automatisk bild-anonymisering (GDPR):** Många CV:n innehåller ett porträttfoto. Om PDF-filen skickas direkt till Claude ser AI-modellen kandidatens ansikte, kön, ålder och etnicitet. Vid lokal konvertering med `pymupdf4llm` filtreras alla bilder bort på millisekunder samtidigt som rubriker, tabeller och kolumnordning bevaras.
3. **Bättre strukturförståelse:** Till skillnad från `PyPDF2` bevarar `pymupdf4llm` rubriknivåer (`#` , `##`) och punktlistor, vilket gör att AI:n enkelt ser skillnad på yrkeserfarenhet, utbildning och sidokolumner.

Den nya 3-stegs-pipelinen per CV

Steg	Komponent / Modell	Vad som sker i steget
Steg 1: För-tvätt (~0,05 sek)	Lokalt i FastAPI (`pymupdf4llm` + Regex)	Konverterar PDF till Markdown, kastar bort alla bilder och raderar e-postadresser, telefonnummer, svenska personnummer samt webbadresser (LinkedIn/GitHub) med reguljära uttryck.

Steg	Komponent / Modell	Vad som sker i steget
Steg 2: AI-Tvätt (~1–2 sek)	Anthropic **Claude Haiku 4.5**	Tar den för-tvättade Markdown-texten och byter ut alla för- och efternamn, fysiska gatuadresser, ålder och könspronomen (han/hon) mot `[BORTTAGET]`. 3x snabbare och ~75 % billigare än Sonnet.
Steg 3: Bedömning (~3–5 sek)	Anthropic **Claude Sonnet 4.5** + Prompt Caching	Utför den stenhårda kompetens- och senioritetsbedömningen mot arbetsannonsen. Arbetsannonsen och systeminstruktionen cachas (`ephemeral`) så att du slipper betala fullt pris för dem vid varje CV.

> **100 % garanterad JSON med Structured Outputs (Pydantic):** Istället för att hoppas att AI:n skriver giltig JSON och rensa svaret med `split("```json")`, använder vi Pydantic-scheman och Tool Use/Structured Outputs i FastAPI. Claude tvingas på protokollnivå att leverera exakt rätt datatyper (`score: int`, `ar_erfarenhet: int`, etc.), vilket eliminerar tolkningsfel helt.

4. Datahantering, Flexibel TTL & Faktureringsmodell

Databasen separerar strikt mellan **permanent affärsdata** (som aldrig innehåller personuppgifter från kandidater) och **temporär analysdata** (som raderas automatiskt).

Flexibel raderingsgräns (TTL – Time-To-Live)

- **7 dagar som standard:** Varje ny analysomgång (och dess krypterade PDF-filer i Supabase Storage) får automatiskt ett utgångsdatum (`expires_at`) satt till 7 dygn framåt i tiden.
- **Användarstyrt datum:** Rekryteraren kan själv välja önskat raderingsdatum i gränssnittet (t.ex. 24 timmar, 7 dagar, 14 dagar, ett specifikt datum eller knappen *"Radera analys direkt"*).
- **Hårt GDPR-tak:** Systemet sätter en maximal gräns i backend (t.ex. 30 eller 60 dagar) så att ingen användare av misstag kan spara kandidatdata på obestämd tid.
- **Automatisk städning:** Ett schemalagt jobb raderar kontinuerligt alla utgångna analyser och tillhörande PDF-filer.

Framtidssäker modell för Limits & Fakturering

Eftersom den slutgiltiga prismodellen (betalning per CV, per rekryteringsannons eller fasta månadspaket) ska kunna anpassas efter hand, bygger vi in två mekanismer från start:

1. **Säkerhetstak / Nödbroms per konto (`cv_quota_limit`):** Varje kundkonto har ett tak i databasen (standard: `200` CV:n för pilotkunder). Innan en analys startar kontrollerar FastAPI att kunden har utrymme kvar på sin kvot. Om gränsen är nådd stoppas körningen med ett tydligt meddelande.
2. **Permanent faktureringslogg (`usage_logs`):** Varje körning loggar en anonymiserad rad med: *Kund-ID, Tidpunkt, Antal analyserade CV:n, Förbrukade Input-tokens och Output-tokens*. Den här tabellen berörs inte när kandidatdatan raderas efter 7 dagar, vilket ger dig ett exakt fakturerings- och marginalunderlag vid månadsslut oavsett vilken prismodell du väljer.

5. Mappstruktur för det nya GitHub-repot (Monorepo)

Vi organiserar både `backend` och `frontend` i samma GitHub-repository (`cv-matcher-saas`) för enkel versionshantering och deploy:

```
cv-matcher-saas/
├── backend/                  # FastAPI (Python)
│   ├── app/
│   │   ├── __init__.py
│   │   ├── main.py           # Startfil för API:et & CORS-inställningar
│   │   ├── config.py         # Miljövariabler (Anthropic, Supabase, GDPR-tak)
│   │   ├── auth.py           # Verifierar inloggad användare via Supabase JWT
│   │   ├── schemas.py        # Pydantic-modeller (Låst JSON-struktur för Claude)
│   │   ├── services/
│   │   │   ├── __init__.py
│   │   │   ├── pdf_parser.py # PDF -> Markdown (pymupdf4llm) + Regex-tvätt
│   │   │   ├── ai_engine.py  # Steg 2 (Haiku-tvätt) & Steg 3 (Sonnet-bedömning)
│   │   │   ├── storage.py    # Krypterad PDF-lagring & TTL-rensning
│   │   │   └── exporter.py   # Genererar Excel-rapporten (.xlsx)
│   │   └── routers/
│   │       ├── __init__.py
│   │       ├── analysis.py   # Endpoints för att starta/hämta/radera analyser
│   │       └── billing.py    # Endpoints för kundens kvot och användningslogg
│   ├── requirements.txt
│   ├── .env.example
│   └── Dockerfile
│
├── frontend/                 # Next.js (TypeScript + Tailwind CSS)
│   ├── src/
│   │   ├── app/
│   │   │   ├── page.tsx      # Publik landningssida
│   │   │   ├── login/        # Inloggningssida (Supabase Auth)
│   │   │   └── dashboard/    # Verktyget: Ny analys, Leaderboard & Kandidatvy
│   │   └── components/       # Återanvändbara UI-komponenter
│   └── package.json
│
├── .gitignore
└── README.md
```

Terminalkommandon för att skapa projektstrukturen (Fas 1)

```
mkdir -p cv-matcher-saas/backend/app/{services,routers} cv-matcher-saas/frontend
cd cv-matcher-saas
touch backend/app/{__init__.py,main.py,config.py,auth.py,schemas.py}
touch
backend/app/services/{__init__.py,pdf_parser.py,ai_engine.py,storage.py,exporter.py
}
touch backend/app/routers/{__init__.py,analysis.py,billing.py}
touch backend/requirements.txt backend/.env.example .gitignore README.md
git init
```

6. Databasschema i Supabase (SQL)

Nedanstående SQL-skript körs i Supabase (EU-region) under *SQL Editor*. Det skapar alla tabeller, aktiverar **Row Level Security (RLS)** så att kunder är 100 % isolerade från varandra, och sätter standard-TTL till 7 dagar:

```
-- 1. KUNDPROFILER & KVOTER (Kopplas automatiskt till Supabase Auth)
CREATE TABLE public.profiles (
    id UUID REFERENCES auth.users(id) ON DELETE CASCADE PRIMARY KEY,
    email TEXT NOT NULL,
    company_name TEXT,
    cv_quota_limit INTEGER NOT NULL DEFAULT 200, -- Säkerhetstak (nödbroms)
    cv_used_count INTEGER NOT NULL DEFAULT 0,    -- Förbrukade CV:n
    default_ttl_days INTEGER NOT NULL DEFAULT 7, -- Standard TTL (7 dagar)
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- 2. FAKTURERINGS- & ANVÄNDNINGSLOGG (Permanent, 0% personuppgifter)
CREATE TABLE public.usage_logs (
    id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
    user_id UUID REFERENCES public.profiles(id) ON DELETE CASCADE NOT NULL,
    cv_count INTEGER NOT NULL,                   -- Antal CV:n i körningen
    input_tokens INTEGER DEFAULT 0,              -- För exakt kostnadskontroll
    output_tokens INTEGER DEFAULT 0,             -- För exakt kostnadskontroll
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- 3. ANALYSOMGÅNGAR (Temporär data med 7 dagars TTL som standard)
CREATE TABLE public.analysis_batches (
    id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
    user_id UUID REFERENCES public.profiles(id) ON DELETE CASCADE NOT NULL,
    title TEXT NOT NULL DEFAULT 'Ny rekrytering',
    job_ad_text TEXT NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    expires_at TIMESTAMPTZ NOT NULL DEFAULT (NOW() + INTERVAL '7 days')
);

-- 4. KANDIDATER I EN ANALYS (Raderas automatiskt med analysomgången)
CREATE TABLE public.candidates (
    id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
    batch_id UUID REFERENCES public.analysis_batches(id) ON DELETE CASCADE NOT
NULL,
    user_id UUID REFERENCES public.profiles(id) ON DELETE CASCADE NOT NULL,
    anonymous_alias TEXT NOT NULL,               -- T.ex. "Kandidat #1"
    score INTEGER NOT NULL,
    years_experience INTEGER,
    education_match TEXT,
    concrete_results TEXT,
    key_skills TEXT,
    missing_requirements TEXT,
    motivation TEXT,
    original_filename TEXT NOT NULL,
    storage_file_path TEXT NOT NULL,             -- Sökväg i Supabase Storage
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- 5. SÄKERHET (Row Level Security - RLS)
ALTER TABLE public.profiles ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.usage_logs ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.analysis_batches ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.candidates ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Egna profiler" ON public.profiles FOR SELECT USING (auth.uid() =
```

7. Exekveringsplan – Byggfaser 1 till 5

Vi bygger och testar applikationen stegvis i fem faser:

Fas	Fokusområde	Konkreta leverabler
Fas 1	Grundstruktur & Databas	Initiera GitHub-repot, skapa mappstrukturen, sätta upp Supabase-projektet i EU och köra SQL-schemat ovan samt skapa en privat Storage-bucket (`cv-vault`).
Fas 2	PDF- & AI-motorn i FastAPI	Bygga `pdf_parser.py` (`pymupdf4llm` + Regex-tvätt), `schemas.py` (Pydantic) och `ai_engine.py` (Haiku 4.5-tvätt + Sonnet 4.5-bedömning med Prompt Caching och asynkron parallellkörning).
Fas 3	Autentisering, Kvoter, Filer & Export	Bygga JWT-verifiering (`auth.py`), kontroll av `cv_quota_limit` , loggning till `usage_logs` , krypterad filhantering med valbar TTL (standard 7 dagar) samt Excel-export (`exporter.py`).
Fas 4	Frontend & Hemsida (Next.js)	Bygga inloggningssida, landningssida och dashboard med uppladdningsvy, val av raderingsdatum (TTL), realtids-progress, Leaderboard, detaljvy per kandidat och knappen "Avslöja & Ladda ner original-CV".
Fas 5	Deploy & Produktionssättning	Driftsätta Backend och Frontend i EU (Render/Vercel), konfigurera automatisk timvis radering av utgångna analyser (TTL-cron) och genomföra skarpa sluttester.